BitFS: A Decentralized Encrypted File System on Blockchain

Version 2.0

Alex Tong
alex@bitfs.org

Abstract

BitFS is a decentralized file system that maps Unix filesystem semantics onto a blockchain-based directed acyclic graph (the Metanet protocol on BSV). Every file is encrypted by default using elliptic-curve Diffie–Hellman key derivation (Method 42), reducing the distinction between private, free, and paid data to a single question: who can derive the decryption key. A hierarchical deterministic wallet mirrors the filesystem tree, yielding stable node identities and deterministic per-file encryption keys. Trustless data commerce is achieved through hash time-locked contracts (HTLC), in which the seller reveals a cryptographic key capsule as the hash preimage to claim payment. The system operates entirely in SPV mode without querying the blockchain. An agent-first interface enables AI agents to discover, navigate, and purchase content autonomously through content negotiation and the HTTP 402 payment protocol.

Contents

1 Introduction

Existing decentralized storage systems face an unresolved tension between data privacy and data commerce. Systems such as IPFS provide content-addressed storage but offer no native encryption or payment mechanism. Filecoin adds economic incentives for storage but requires computationally expensive zero-knowledge proofs and provides weak incentives for content retrieval. Centralized cloud storage offers convenience but demands trust in a single custodian for both data integrity and access control.

These limitations grow sharper as AI agents emerge as autonomous participants in the digital economy. An agent that discovers a dataset behind a paywall cannot complete a credit card transaction, negotiate a license agreement, or solve a CAPTCHA. The existing payment infrastructure of the web was designed for humans, and the gap is widening.

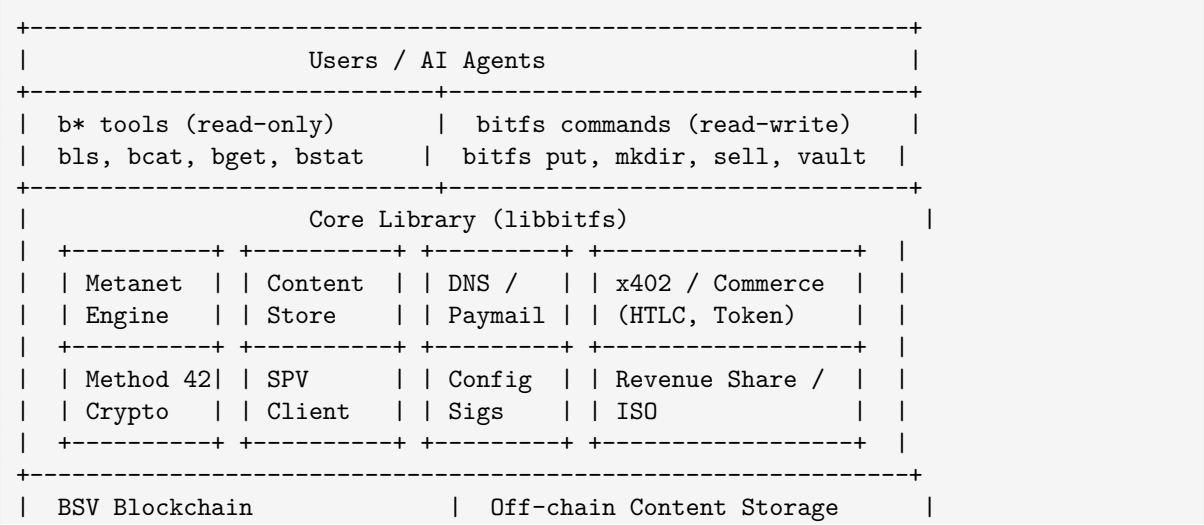
BitFS addresses these tensions through three observations. First, metadata and content require different guarantees: metadata benefits from blockchain immutability and global ordering, while file content requires efficient storage and retrieval. The two should be separated. Second, encryption should be universal rather than optional. When all data is encrypted by default, the distinction between public and private reduces to key distribution; public data simply uses a trivially derivable key. Third, AI agents are first-class users. A file system designed for the coming decade must expose machine-readable interfaces that enable agents to discover content, negotiate payment, and transact without human intermediation.

BitFS realizes these ideas as a Unix-style filesystem where inodes are blockchain public keys, directory entries are Metanet transaction payloads, and all file content is encrypted using deterministic ECDH key derivation. A single cryptographic framework, Method 42 [1], provides file encryption, access control, identity authentication, and trustless commerce simultaneously.

Content distribution is handled by the Metanet network [5], a decentralized CDN that incentivizes Metanet Nodes to serve data through retrieval fees rather than storage subsidies. BitFS defines the file system protocol; the Metanet network provides the distribution layer. Their relationship is analogous to that of IPFS and Filecoin, but with one key difference: Metanet incentivizes retrieval (serving data) rather than storage (hoarding data), aligning economic incentives with user experience.

2 System Architecture

The BitFS architecture separates concerns into distinct layers:



```
|  (Metanet metadata, HTLC)    |  (Daemon / LFCP service)       |
+------------------------------+--------------------------------+
|  Metanet Network (Decentralized CDN)                          |
|  (Incentivized retrieval, storage contracts, merged mining)   |
+---------------------------------------------------------------+
```

The top layer presents two classes of command-line tools following the Unix philosophy: read-only tools (prefixed `b*`) for visitors who need no wallet, and read-write commands (under `bitfs`) for content owners. The core library implements Metanet DAG traversal, Method 42 encryption, SPV verification, DNS and Paymail resolution, and the HTLC commerce protocol.

The bottom two layers handle persistence. The BSV blockchain stores Metanet metadata (directory structure, access policies, content hashes, pricing), while actual file content resides either locally on the daemon of the owner or on the Metanet decentralized CDN. This separation ensures that metadata benefits from blockchain-grade immutability while content storage remains flexible and efficient.

Each filesystem operation produces a Metanet transaction with a self-sustaining UTXO structure. One output refreshes the spendable UTXO of the parent node, enabling the next operation without pre-funding node addresses. Initial funding comes solely from a shared fee keychain. The filesystem can therefore grow indefinitely without per-node funding setup, a property that matters for systems managing thousands of files.

3 Metanet DAG as Filesystem

The Metanet protocol [2] defines a directed acyclic graph on the BSV blockchain where each node is a transaction containing an `OP_RETURN` output with a node public key P_{node} and a parent transaction reference. Edges are cryptographically authenticated: a child transaction must include an input signed by the private key D_{parent} of the parent, which the network verifies as part of standard transaction validation. No additional trust assumptions are needed beyond the existing signature verification of Bitcoin. The permission model is native to the blockchain: only the holder of D_{parent} can create child nodes, and every miner on the network enforces this rule.

BitFS maps Unix filesystem concepts onto this structure directly:

Unix concept	**BitFS equivalent**
inode	P_{node} (compressed public key, 33 bytes)
directory entry	`ChildEntry(index, name, type, pubkey)`
inode number	index (monotonic auto-increment per directory)
parent directory	parent field in payload (P_{node} of parent)

A filename exists only in the child entry list of the parent directory, not in the node itself, exactly as in Unix, where an inode contains no name. Three node types are defined. A FILE node holds a content hash, MIME type, and file size. A DIR node holds a children list and a counter for the next available child index. A LINK node holds a target reference in one of two forms: SOFT (pointing to a P_{node}, always resolving to the latest version) or SOFT_REMOTE (pointing to a domain and path for cross-user references resolved via DNS).

Hard links are implemented implicitly: multiple directory entries in possibly different directories reference the same P_{node}. This is not a LINK node type but a property of directory entries—two entries sharing a public key share the underlying file data. Soft links introduce an indirection

layer, always resolving to the latest version of the target. Remote soft links extend this across trust boundaries, referencing a domain and path that resolve through DNS, enabling cross-user file references without requiring both parties to share a Metanet tree.

Version control is intrinsic to this model. The same P_{node} may appear in multiple transactions, each representing a version of the file or directory. The transaction with the greatest block height (or latest topological order within a block) is the current version. All previous versions remain accessible and immutable on-chain, providing a complete audit trail without any additional versioning mechanism. This property also enables BitFS to serve as a git remote: a custom git remote helper translates push and fetch operations into Metanet transactions, storing encrypted packfiles as BitFS file nodes. The same access control and commerce features available for ordinary content apply equally to code repositories.

4 Hierarchical Key Derivation

The identity key P_{node} of each node is derived from a BIP32 [3] hierarchical deterministic wallet, structured to mirror the filesystem hierarchy:

```
m/44'/236'/0'             Fee keychain (shared across all vaults)
m/44'/236'/1'/0/0         Vault 0 root directory
m/44'/236'/1'/0/0/1       First child of root
m/44'/236'/1'/0/0/1/3     Third child of first child
m/44'/236'/2'/0/0         Vault 1 root directory (independent tree)
```

The BIP32 derivation path mirrors the filesystem path. The root directory of a vault is at derivation index `/0/0` under its account. When a new file is created in a directory, it receives the next available child index, which becomes both its directory entry number and the final segment of its BIP32 derivation path.

A *Vault* is an independent Metanet tree rooted at a BIP32 account. Multiple vaults share a single seed but maintain entirely separate directory hierarchies, each capable of binding to its own domain name. A single user can thus maintain personal files, a company website, and a public dataset as isolated trees, all recoverable from one BIP39 mnemonic.

This design provides two important properties. First, *stable node identity*: the P_{node} of a directory does not change when its contents are updated, enabling persistent references and reliable soft links. Second, *deterministic recovery*: the entire key hierarchy can be regenerated from the mnemonic seed, allowing the owner to reconstruct their cryptographic identity even if local state is lost.

5 Universal Encryption

All files are encrypted before storage. Rather than treating encryption as optional, BitFS encrypts everything and uses key derivation rules to determine who can decrypt. The encryption scheme is based on Method 42 [1], an ECDH construction over the secp256k1 curve, the same curve used by Bitcoin for transaction signatures. No additional cryptographic infrastructure is needed.

The encryption key for each file is derived as follows:

1. shared_point = ECDH(D_{node}, $P_{\text{recipient}}$) = $D_{\text{node}} \times P_{\text{recipient}}$
2. key_hash = SHA256(SHA256(plaintext))

3. aes_key = HKDF-SHA256(ikm = shared_point.x, salt = key_hash, info = `"bitfs-file-encryption"`
4. ciphertext = AES-256-GCM(plaintext, aes_key)

Here D_{node} is the BIP32-derived private key of the node and $P_{\text{recipient}}$ is the public key of the intended reader. The key_hash serves two purposes: it is the salt for key derivation and a content integrity commitment recorded in the Metanet metadata. Using the double hash SHA256(SHA256(plaintext)) prevents exposing the raw content hash on-chain while still allowing post-decryption verification.

Three access levels emerge from this single mechanism without any changes to the encryption scheme itself:

Private: The recipient is the owner. ECDH($D_{\text{node}}, P_{\text{node}}$) produces a shared secret that only the owner can compute. The entire Metanet payload is encrypted, hiding filenames, sizes, timestamps, and directory structure.

Free: The "trivial key trick" sets $D_{\text{node}} = 1$, making the shared point simply $P_{\text{recipient}}$ (the scalar one times any point yields the point itself). Since P_{node} is published via DNS and the recipient is set to the node itself, the shared point equals P_{node} and anyone can compute the decryption key. The data remains encrypted on disk, maintaining a uniform storage model, but the key is publicly derivable.

Paid: Standard ECDH between the D_{node} of the owner and the P_{buyer} of the buyer. The buyer obtains the shared secret (called a key capsule) through an HTLC atomic swap, as described in Section 7.

This unified approach means the storage layer handles only opaque encrypted byte sequences regardless of access policy. There is no "unencrypted" mode that might leak data through misconfiguration. The system defaults to encryption; accessibility is determined entirely by key distribution.

6 Content-Addressed Storage

BitFS separates metadata from content. Metanet transactions on the blockchain store only metadata: directory structure, content hashes, access policies, pricing, and timestamps. The actual encrypted file content is stored independently in a content-addressed store indexed by key_hash.

This separation provides several benefits. Metadata updates (renaming a file, changing its price, updating access policy) do not require re-uploading the content. The same encrypted content can be referenced by multiple metadata entries. The implementation of the content store can vary (local filesystem, the daemon of the owner, the Metanet CDN, or even on-chain via `OP_DROP` transactions) without changing the metadata protocol.

The daemon of the owner serves content over standard HTTP. A request to `GET /data/{hash}` returns the encrypted bytes. Since all content is encrypted, the storage layer requires no access control of its own. Deduplication occurs naturally: two files with identical ciphertext share a single stored copy. No external dependency such as IPFS is required.

For content that must be permanently preserved on-chain, BitFS supports an optional on-chain storage mode where encrypted content is embedded in separate data transactions using `OP_DROP`. Large files are split across multiple transactions with a recombination hash for integrity verification. This mode is suited to small, high-value content that benefits from blockchain-grade permanence.

7 Trustless Data Commerce

Paid content is purchased through a hash time-locked contract (HTLC) atomic swap. The protocol requires no trusted intermediary, no buyer database, and no session state on the side of the seller.

The process begins with a Method 42 ECDH handshake for mutual authentication. The buyer and seller exchange public keys and nonces, then independently compute a shared session key:

$$\text{session\_key} = \text{SHA256}(\text{ECDH}(D_{\text{self}}, P_{\text{other}}).x \parallel \text{nonce}_{\text{buyer}} \parallel \text{nonce}_{\text{seller}})$$

The public key of the seller must match the P_{node} published via DNS, preventing man-in-the-middle attacks. Only the holder of the corresponding private key can compute the correct session key.

After authentication, the seller computes a key capsule (the ECDH shared secret between the private key of the owner and the public key of the buyer) and returns its hash (`capsule_hash`) along with a payment address. The buyer creates an HTLC transaction on the blockchain:

```
IF SHA256(preimage) == capsule_hash AND Sig(seller)  ->  seller claims
ELSE IF timeout_expired AND Sig(buyer)               ->  buyer refunds
```

The atomic swap guarantees that the seller receives payment if and only if the buyer receives the key capsule. When the seller reveals the capsule (as the HTLC preimage) to claim payment, the buyer reads the capsule from the blockchain and uses it to derive the decryption key for the file. Subsequent access requires only the cached key and the encrypted content; no further interaction with the seller is needed.

For directory-level purchases (planned), BIP32 key derivation provides a natural solution. Because ECDH is algebraically compatible with the non-hardened derivation of BIP32, a single capsule for a parent directory can unlock all non-hardened child nodes:

$$S_{\text{child}} = S_{\text{parent}} + \text{offset} \times P_{\text{buyer}}$$

where *offset* is derived from the extended public key (xpub) of the parent and the index of the child. The buyer needs only one HTLC transaction to purchase an entire directory tree. The seller provides the xpub of the parent directory alongside the capsule, and the buyer derives all child decryption keys locally.

This mechanism also provides a natural access control boundary. Nodes created with hardened BIP32 derivation cannot be derived from the capsule of the parent and require separate purchase. Content owners can therefore offer a directory for sale while excluding premium items that must be purchased individually. Consider a magazine publisher who sells a monthly subscription as a directory purchase. Regular issues are created with non-hardened derivation and are automatically accessible to subscribers. Special editions use hardened derivation and require a separate HTLC. The access policy is encoded in the key derivation structure itself, not in application-level logic.

For high-volume purchasers, a hash chain token system (planned) will provide batch efficiency. A buyer pre-purchases N access tokens through a single on-chain setup, then redeems them one at a time for individual file keys. Each token in the chain validates against the previous one (T_i satisfies $H(T_i) = T_{i-1}$), providing a cryptographic guarantee of the integrity of the chain

without requiring N separate HTLC transactions. The single-file HTLC, directory-tree xpub purchase, and hash chain token system coexist as three commerce modes suited to different access patterns: occasional purchases, bulk directory access, and high-frequency subscriptions respectively.

8 SPV Operation

BitFS operates entirely in Simplified Payment Verification mode [4]. The owner stores all self-created transactions and their Merkle proofs locally. Visitors obtain metadata from the daemon of the owner, never from the blockchain directly.

A visitor verifies received metadata through a complete SPV verification chain: confirm that the transaction is well-formed, verify its Merkle proof against the block header, check that the block header belongs to the longest chain, and after decryption, verify that SHA256(SHA256(plaintext)) matches the key_hash committed in the Metanet payload. If any step fails, the data is rejected.

This design eliminates dependence on blockchain indexing services for normal operation. The daemon of the owner is the canonical source of truth for its own data, and any claim it makes can be independently verified through Merkle proofs without trusting the daemon itself. A third-party index serves only as a degraded fallback when the daemon of the owner is unreachable.

Recovery from the BIP39 seed restores the entire HD key hierarchy, allowing the owner to reconstruct their cryptographic identity. Transaction data must be restored from backup, as the system deliberately never scans the blockchain.

This peer-to-peer verification model has a useful property: the data itself carries its proof of authenticity. A Metanet payload accompanied by a valid Merkle proof and block header is self-certifying. It can be passed from peer to peer, cached by CDN nodes, or stored offline, and any recipient can independently verify its integrity and provenance. No connection to the original publisher or to any blockchain node is required at verification time.

9 Agent-First Interface

BitFS treats AI agents as first-class users. The daemon implements HTTP content negotiation to serve both humans and agents from the same endpoints. When a request arrives with `Accept: text/html`, the daemon returns an HTML page with embedded WebMCP tool declarations (planned) that browser-based agents will discover through `navigator.modelContext`. When the accept header is `text/markdown`, the response is a self-describing usage guide with CLI command templates. JSON responses provide structured metadata for programmatic consumption.

For paid content, the HTTP 402 response includes payment metadata headers (price, price-per-KB, file size, invoice ID, expiry) alongside a format-appropriate body. A human sees a paywall. A browser-based agent discovers WebMCP tool declarations and can invoke payment autonomously. A CLI agent receives a concrete command template and executes it.

The 402 status code, traditionally a dead end for automated clients, becomes a programmable payment API when the client has a wallet. An agent encountering a 402 response can read the price, verify it against a budget policy, construct an HTLC transaction, broadcast it, receive the key capsule, decrypt the content, and proceed—all without human involvement. The same protocol flow that presents a paywall to a browser becomes a transparent micropayment for an agent.

The dual-mode CLI reflects this philosophy. Read-only tools (`bls`, `bcat`, `bget`, `bstat`, `btree`)

are stateless and require no wallet, serving the visitor role. They mirror familiar Unix commands: `bls` lists directories like `ls`, `bcat` outputs file content like `cat`, `bget` downloads files like `wget`. Read-write commands (under `bitfs`) require a wallet and serve the owner role: `bitfs put`, `mkdir`, `rm`, `mv`, `cp`, `link` for filesystem operations; `bitfs sell`, `encrypt`, `decrypt` for commerce and access control; `bitfs vault`, `wallet`, `publish`, `daemon` for infrastructure management. An FTP-style interactive shell (`bitfs shell`) provides a REPL for navigating the on-chain filesystem with familiar commands such as `cd`, `ls`, `get`, `put`, and `lcd`.

All tools produce JSON output with the `--json` flag, making them composable in shell pipelines and accessible to programmatic callers. The read-only tools support `--offline` mode using locally cached metadata, enabling continued operation when the network is unavailable. Pricing information uses a simple `price_per_kb` field (satoshis per kilobyte) that supports directory inheritance: a file without an explicit price inherits from its nearest ancestor that defines one, allowing an entire directory tree to be priced with a single setting.

10 Identity and Discovery

BitFS provides three addressing modes that coexist and resolve through a unified URI scheme.

DNS binding associates any Metanet node with a domain name through two DNS records: a TXT record at `_bitfs` containing the public key of the node in the format `bitfs=<hex pubkey>`, and one or more SRV records at `_bitfs._tcp` specifying service endpoints with priority and weight for load balancing. Bidirectional verification ensures consistency: the DNS record points to P_{node}, and the domain field of the Metanet payload records the bound domain. Both must agree.

Paymail integration [6] adds email-style addressing. The URI `bitfs://alice@example.com/docs/paper.pd` routes through the Paymail protocol to resolve the alias "alice" at the domain "example.com" to the root public key of a specific vault. This enables multiple users under a single domain, a capability that DNS binding alone cannot provide, since the TXT record of a domain can point to only one P_{node}. The BitFS daemon simultaneously serves as a Paymail server, exposing PKI, public profile, and key verification capabilities.

Direct public key addressing (`bitfs://02a1b2c3.../path`) bypasses all discovery mechanisms for cases where the identity of the node is already known.

URI resolution follows a deterministic precedence: the presence of `@` triggers Paymail resolution, a hexadecimal authority triggers direct key addressing, and anything else triggers DNS lookup. In all cases, the result is a P_{node} and a set of service endpoints, followed by a Method 42 handshake, Metanet metadata retrieval via SPV, and directory tree traversal.

```
bitfs://alice@example.com/docs/paper.pdf   Paymail (@ present)
bitfs://example.com/docs/paper.pdf         DNS (no @, not hex)
bitfs://02a1b2c3.../docs/paper.pdf         Direct key (hex pubkey)
```

The Paymail layer bridges human-readable identity and the cryptographic identities used on-chain. It enables scenarios such as displaying `bob@handcash.io` as a buyer identity in sales records rather than a 33-byte public key, looking up the public key of a collaborator by email-style address for Method 42 handshakes, and using different Paymail aliases to route to different vaults under the same domain. Paymail operates purely at the discovery layer. The on-chain protocol uses only raw public keys; Paymail is never embedded in blockchain transactions or Metanet payloads.

11 Revenue Rights *(Planned)*

Note: This section describes a planned feature. The `revshare` *package exists in libbitfs-go but is not yet implemented.*

BitFS enables content creators to tokenize the revenue rights of their files. Revenue rights are represented as a dual-layer on-chain structure: Share UTXOs serve as bearer instruments proving ownership of a fraction of the future earnings of a file, while a Registry UTXO acts as a stateful covenant maintaining the canonical list of shareholders and their proportions.

An Initial Share Offering (ISO) allows creators to sell revenue shares to the public. The creator defines a total share count (analogous to a company choosing how many shares to issue), reserves a portion, and places the remainder in an ISO Pool, a special covenant that acts as an automated market maker, accepting payment from any buyer and issuing Share UTXOs atomically. The ISO Pool enforces price, verifies payment, updates the Registry, and issues shares in a single transaction with no trusted intermediary.

When a file is purchased through HTLC, the payment is automatically distributed to all shareholders according to their registered proportions. The Registry covenant reads the current shareholder list and enforces the correct split in the transaction outputs. Share UTXOs can be freely transferred, split, or merged on the secondary market through atomic swaps—single transactions where share ownership and payment change hands simultaneously, with no possibility of one side defaulting.

This system transforms digital content from a product sold for a one-time fee into an income-generating asset. A musician can sell 60% of the future earnings of a song through an ISO, retaining 40%. Investors who purchase shares earn a proportional cut of every future sale. The entire lifecycle (issuance, purchase, transfer, and revenue distribution) is enforced on-chain by covenant scripts, requiring no trust in any party.

12 The Metanet Network

BitFS defines the file system protocol: how files are organized, encrypted, addressed, and traded. Content distribution—the question of how encrypted bytes travel from storage to the requesting client—is designed to be handled by the Metanet network, a decentralized CDN described in its own whitepaper [5]. Integration between the two systems is under active development.

The Metanet network is a BSV-homomorphic sidechain that uses the same transaction format and script engine as BSV, with its own genesis block, native token (MNT), and merged mining with SHA256 proof-of-work. Metanet Nodes on the network earn revenue by serving content to users, collecting x402 retrieval fees in BSV. This creates a positive feedback loop: popular content generates more retrieval fees, attracting more nodes to cache it, which in turn improves availability and reduces latency.

For content owners, integration is simple. A planned `--store metanet` flag for `bitfs put` will publish files to the decentralized CDN. Metanet Nodes that cache the content share x402 revenue with the owner according to a configurable split. Cold or unpopular data can be preserved through explicit storage contracts where the owner pays nodes in MNT tokens. In both cases, the storage proof mechanism reuses Method 42: each copy held by a node is re-encrypted with a provider-specific ECDH key, making copies cryptographically unique and verifiable through simple Merkle challenges rather than expensive zero-knowledge proofs.

Ordinary users interact only with BSV. The MNT token economy operates between content owners and Metanet Nodes, invisible to end users and agents who simply request files and pay

x402 fees in BSV.

13 Conclusion

BitFS demonstrates that a Unix-style file system can be built on blockchain primitives without sacrificing the properties that make filesystems useful: hierarchical organization, stable references, version history, and familiar command-line semantics. The design centers on a single cryptographic construction, ECDH key derivation on secp256k1, which simultaneously provides file encryption, access control tiering (private, free, paid), mutual authentication, trustless atomic commerce, and content integrity verification.

By treating all data as encrypted by default, BitFS eliminates the configuration surface area where privacy failures typically occur. By mapping BIP32 key derivation onto the filesystem tree, it enables directory-level purchases and subscription models through the algebraic properties of non-hardened child key derivation. By operating in SPV mode, it avoids dependence on blockchain indexing infrastructure. By designing for AI agents as first-class users, it transforms the HTTP 402 paywall from a barrier into a programmable payment interface.

The separation of BitFS (the filesystem protocol) from the Metanet network (the distribution layer) allows each to evolve independently while maintaining a clean interface: BitFS produces encrypted, content-addressed data with on-chain metadata; the Metanet network distributes that data with economic incentives aligned toward retrieval performance rather than storage capacity.

Future extensions include group signatures for multi-user write access (using BBS+ on BLS12-381 curves), sCrypt-based on-chain verification of key capsules for fully automated commerce without seller presence, and time-locked access using `OP_CHECKLOCKTIMEVERIFY` for subscription expiration, scheduled publication, and time-bounded permissions. These capabilities build on the existing cryptographic and transaction framework without requiring protocol-level changes.

References

[1] C. S. Wright, "An Immutable File and Data Store," nChain, 2019. Method 42: Deterministic key derivation for per-file encryption using ECDSA/secp256k1.

[2] nChain, "The Metanet Technical Summary v1.0," 2020. Blockchain-based directed acyclic graph for organizing data relationships.

[3] P. Wuille, "BIP32: Hierarchical Deterministic Wallets," Bitcoin Improvement Proposals, 2012.

[4] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Section 8: Simplified Payment Verification.

[5] GB2608179A, "Multi-level Blockchain," UK Intellectual Property Office, 2021. Embedding secondary data chains on a core blockchain.

[6] BSV Blockchain, "Paymail (bsvalias) Protocol," BSV Association. Email-style addressing for BSV identity and payment resolution.